



تمرین شماره ۲



عنوان: Physical Unclonable Functions

درس: امنیت سخت افزار

استاد: دکتر سیامک محمدی

دستیاران آموزشی: محمدرضا ولی^۱

نیمسال دوم سال تحصیلی ۱۴۰۳-۰۴

تحلیل Physical Unclonable Functions با استفاده از PyPUF

در این تمرین می‌خواهیم رفتار Arbiter PUF و XOR Arbiter PUF را با استفاده از [PyPUF](#) شبیه‌سازی کرده و نحوه تولید Challenge-Response Pairs (CRPs) را در آن‌ها بررسی کنیم. همچنین مقادیر معیارهای Reliability, Uniqueness و Bit Aliasing را برای این دو PUF ارزیابی کرده و اثر نویزهای محیطی را بر آن‌ها مشاهده می‌کنیم. در انتها با شبیه‌سازی Machine learning modeling attack امنیت PUF‌ها را ارزیابی و مقایسه و Randomness خروجی تولید شده به عنوان یک کلید توسط دو PUF را بررسی خواهیم کرد.

برای شروع انجام این تمرین ابتدا با وارد کردن دستور زیر در ترمینال خود از نصب ابزارهای لازم اطمینان حاصل کنید:

```
pip install pypuf numpy matplotlib scikit-learn
```

دستورات و کامنت‌هایی که در هر بخش در فایل تمرین مشاهده می‌کنید، صرفاً برای درک بهتر مراحل تمرین مطرح شده‌اند بنابراین لازم است صحت کد خود را در هر مرحله طبق مستندات PyPUF بررسی کنید.

۱. نمونه‌سازی [Arbiter PUF](#) و تولید CRP Table

یک فایل با نام arbiter_puf.py ایجاد کنید و تعداد Stage طراحی خود را برابر 64 قرار دهید.

```
from pypuf.simulation import ArbiterPUF
# Instantiate an Arbiter PUF with 64 stages
n_stages = 64
puf = ArbiterPUF(n=n_stages)
print("Arbiter PUF instantiated with", n_stages, "stages.")
```

حال لازم است یک فایل جدید با عنوان generate_crps.py ایجاد کرده و اقدام به ساخت تعدادی CRP نمایید (در این مرحله تعداد CRP‌ها را برابر ۱۰۰۰۰ در نظر بگیرید). خروجی این مرحله می‌تواند به یکی از دو صورت زیر باشد:

۱. فایل csv شامل Challenge‌ها با عنوان challenges.csv به همراه یک فایل csv شامل Response‌های متناظر با عنوان responses.csv
۲. فایل csv شامل CRP table

همچنین توصیه می‌شود تمام Response‌هایی که در فایل csv شما نمایش داده می‌شوند را به صورت یک String شامل تمام Response‌ها نیز استخراج کنید (این String در مرحله ۶ برای بررسی Randomness مورد استفاده قرار می‌گیرد)

```
import numpy as np
from pypuf.simulation import ArbiterPUF
n_stages = 64
```

```
num_crps = 10000 # Number of challenge-response pairs to generate
puf = ArbiterPUF(n=n_stages)

# Generate random binary challenges.
challenges = np.random.randint(0, 2, size=(num_crps, n_stages))
responses = puf.eval(challenges)

# Save challenges and responses to CSV files.
np.savetxt("challenges.csv", challenges, fmt="%d", delimiter=",")
np.savetxt("responses.csv", responses, fmt="%d", delimiter=",")
print("Generated", num_crps, "CRPs. Files 'challenges.csv' and 'responses.csv' have been saved.")
```

۲. ارزیابی [Reliability](#)، [Uniqueness](#) و [Bit Aliasing](#)

برای ارزیابی Reliability (Intra-chip Hamming distance) یک فایل به نام `evaluate_reliability.py` ایجاد کنید و کد زیر را وارد نمایید:

```
import numpy as np
from pypuf.simulation import ArbiterPUF
from sklearn.metrics import pairwise_distances

n_stages = 64
num_crps = 1000 # Use a subset for faster computation
puf = ArbiterPUF(n=n_stages)

#Generate challenges.
challenges = np.random.randint(0, 2, size=(num_crps, n_stages))

#Evaluate the same challenge set twice.
responses1 = puf.eval(challenges)
responses2 = puf.eval(challenges)

#Calculate Hamming distances (as a fraction).
distances = np.mean(responses1 != responses2)
print("Average intra-chip Hamming distance:", distances)
```

حال برای ارزیابی Uniqueness (Inter-chip Hamming distance) یک فایل جدید به نام `evaluate_uniqueness.py` ایجاد کرده و کد زیر را وارد کنید:

```
import numpy as np
from pypuf.simulation import ArbiterPUF
n_stages = 64
num_crps = 1000

#Create two distinct PUF instances.

puf1 = ArbiterPUF(n=n_stages, seed=1)
puf2 = ArbiterPUF(n=n_stages, seed=2)
challenges = np.random.randint(0, 2, size=(num_crps, n_stages))
responses1 = puf1.eval(challenges)
responses2 = puf2.eval(challenges)

#Compute average Hamming distance between responses of the two PUFs.

uniqueness = np.mean(responses1 != responses2)
print("Average inter-chip Hamming distance:", uniqueness)
```

در ادامه برای ارزیابی Bit Aliasing (Uniformity) یک فایل جدید به نام `evaluate_uniformity.py` ایجاد کرده و طبق توضیحات این [لینک](#) اقدام به بررسی مقادیر این معیار نمایید.

۳. نویز

تا این مرحله تمام شبیه‌سازی‌ها در شرایط ایده آل انجام شده‌اند در حالی که در دنیای واقعی، نویزهای محیطی می‌توانند منجر به Temporal variations شوند که نتیجه آن عدم سازگاری احتمالی Challenge‌ها با Response‌های متناظر و کاهش Reliability می‌باشد. در این مرحله قصد داریم اثر نویز را بر CRP table بررسی می‌کنیم.

به این منظور یک فایل به نام `simulate_noise.py` ایجاد کنید و کد زیر را وارد نمایید:

```
import numpy as np
from pypuf.simulation import ArbiterPUF
import matplotlib.pyplot as plt
n_stages = 64
num_crps = 1000
```

```
puf = ArbiterPUF(n=n_stages)
challenges = np.random.randint(0, 2, size=(num_crps, n_stages))
responses_clean = puf.eval(challenges)

#Introduce noise: flip bits with a given probability.
noise_level = 0.05 # 5% noise
noise = np.random.rand(*responses_clean.shape) < noise_level
responses_noisy = np.bitwise_xor(responses_clean, noise.astype(int))

#Calculate average Hamming distance between clean and noisy responses.
noise_effect = np.mean(responses_clean != responses_noisy)
print("Average Hamming distance due to noise:", noise_effect)

#Plot a small segment of clean vs noisy responses.
plt.figure(figsize=(10, 3))
plt.plot(responses_clean[:50], 'b.-', label="Clean")
plt.plot(responses_noisy[:50], 'r.-', label="Noisy")
plt.legend()

plt.title("Comparison of Clean and Noisy Responses (First 50 CRPs)")
plt.xlabel("Challenge Index")
plt.ylabel("Response Bit")
plt.show()
```

۴. Machine learning modelling attack

الگوریتم‌های مختلف یادگیری ماشین می‌توانند به عنوان ابزاری برای شبیه‌سازی رفتار یک طراحی PUF و ساختن یک مدل برای پیش‌بینی CRP Table ها و در نتیجه به خطر انداختن امنیت آن طراحی مورد استفاده قرار گیرند. در این بخش می‌خواهیم یکی از انواع این حملات بر اساس [Logistic Regression](#) را بررسی کنیم. یک فایل به نام `ml_attack.py` ایجاد کنید و `CRP table` مرحله ۲ را هدف حمله قرار دهید.

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

#Load the previously saved CRPs.
```

```

challenges = np.loadtxt("challenges.csv", delimiter=",", dtype=int)
responses = np.loadtxt("responses.csv", delimiter=",", dtype=int)

#Split data: 70% training, 30% testing.

X_train, X_test, y_train, y_test = train_test_split(challenges, responses, test_size=0.3,
random_state=42)

#Train a logistic regression model.

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

#Predict on test set and evaluate accuracy.

predictions = model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)
print("Machine Learning Attack Accuracy:", accuracy)

```

۵. [XOR Arbiter PUF](#)

تمام مراحل قبلی را در رابطه با XOR Arbiter PUF نیز تکرار کنید (با استفاده از معیارهای مستند شده در گزارش نهایی) و نتایج آن را با نتایج مراحل قبلی در رابطه با Arbiter PUF مقایسه کنید.

۶. تحلیل تصادفی بودن Response ها و کاربرد PUF به عنوان TRNG

یکی از کاربردهای مهم PUF ها استفاده از Response های تولید شده به عنوان اعداد تصادفی و در نتیجه استفاده از PUF به عنوان یک TRNG مستقل یا منبعی برای افزایش Randomness در سایر RNG ها می باشد. فرض کنید می خواهیم از یک String شامل تمام Response های جمع آوری شده در مرحله ۲ (و به همین ترتیب در مرحله ۵) به عنوان دو کلید منحصر به فرد برای استفاده در یک برنامه احراز هویت استفاده کنیم. از آنجایی که هر قدر Randomness مقادیر بیت های این کلید بیشتر باشد، احتمال شکستن آن توسط یک شخص مهاجم کمتر می شود، در این بخش می خواهیم میزان تصادفی بودن کلیدهای به دست آمده خود را بسنجیم.

برای تست Randomness کلید خود از سایت [Random Bitstream Tester](#) استفاده می کنیم که امکان تحلیل نتایج ۱۴ تست از میان ۱۵ تست مجموعه تست های NIST SP 800-22 را فراهم می کند. مجموعه آزمون های تست تصادفی بودن NIST SP 800-22 یک استاندارد است که توسط مؤسسه استانداردها و فناوری ملی (NIST) منتشر شده و به منظور ارزیابی کیفیت ژنراتورهای اعداد تصادفی (RNG) مورد استفاده قرار می گیرد. این مجموعه شامل ۱۵ تست مختلف است که هر کدام به جنبه های متفاوتی از تصادفی بودن داده ها می پردازند، از جمله:

- آزمون فراوانی (Frequency Test): بررسی می کند که آیا تعداد بیت های ۰ و ۱ تقریباً برابر است یا خیر
- آزمون بلاک فراوانی (Block Frequency Test): بررسی توزیع بیت ها در بخش های کوچک داده
- آزمون سری (Runs Test): ارزیابی تعداد سری های پیوسته بیت های ۰ یا ۱

- **آزمون پیچیدگی خطی (Linear Complexity Test):** اندازه‌گیری دشواری تولید یک دنباله خطی که خروجی داده‌ها را تولید کند

این آزمون‌ها به شما کمک می‌کنند تا سطح تصادفی بودن یک دنباله بیتی را با دقت بسنجید. ابزار Random Bitstream Tester از ۱۴ آزمون این مجموعه استفاده می‌کند و نتایج هر آزمون به شما نشان می‌دهد که آیا دنباله ورودی از نظر آماری تصادفی است یا خیر. در صورت علاقه به کسب اطلاعات بیشتر درباره NIST SP 800-22 می‌توانید به [مستندات رسمی](#) آن مراجعه نمایید.

برای انجام این تحلیل لازم است ابتدا وارد سایت شده، از بخش **Select the input type** گزینه **Manual bitstream input** را انتخاب کرده و **String** تولید شده خود را وارد کنید تا نتایج تحلیل تست‌ها را دریافت کنید. از خروجی تست‌ها یک اسکرین شات گرفته و آن را به همراه توضیحاتی مختصر درباره ۵ تست از میان این ۱۴ تست و تحلیل **Randomness** هر کلید در گزارش خود قید نمایید.

ملاحظات تمرین

مهلت تحویل: ۱۹ اردیبهشت

- تکلیف به صورت انفرادی انجام می شود.
- به ازای هر روز تاخیر ۱۰ درصد از نمره شما کسر میشود.
- تقلب نمره منفی ۱۰۰ را برای همه طرفین در پی دارد.
- خروجی های عددی شما می توانند به صورت عدد، درصد یا گراف باشند.
- لطفا پاسخ خود را شامل کدها، گزارش شامل مراحل انجام تمرین و تحلیل نتایج به دست آمده در رابطه با دو طرح PUF بررسی شده در این تمرین را در قالب یک فایل فشرده با فرمت نام گذاری زیر در سامانه Elearn بارگذاری کنید:

StudentID_NameLastname_HWX

- می توانید سوالات خود را از طریق آیدی های تلگرام زیر مطرح کنید:

@jxteler

موفق باشید.